RouteWise

Detaylı Sistem Analizi ve Akış Şeması

Lojistik Dağıtım Süreçlerinde Dinamik Rota ve Araç Optimizasyonu Sağlayan Akıllı Yazılım Sistemi

Kapsam: Bandırma içi kargo dağıtımı — tek dağıtım merkezi

Problem sınıfı: CVRP + maksimum rota süresi + dinamik yeniden rotalama

Yığın: Spring Boot · React · PostgreSQL/PostGIS · Python

Repo: github.com/AllenVB/RouteWise

Sürüm: Taslak v1 · 10.10.2026 — *revizyon için*

Bu doküman revizyon içindir. Veri modeli ve bazı iş kuralları bu çalışmada ilk kez tasarlanmıştır; **varsayım** olarak işaretlenen noktalar danışman görüşüne açıktır ve değiştirilebilir.

Tanım ve Yapı

02 Problem tanımı, kapsam ve kısıtlar
03 Teknoloji yığını — kullanılan her şey
04 Sistem mimarisi ve bileşenler
05 Docker dağıtım mimarisi

Veri Modeli

06 ER diyagramı — genel görünüm
07 Tablo detayları I — kullanıcı, rol, adres
08 Tablo detayları II — kargo, rota, optimizasyon
09 İlişki kardinaliteleri ve bütünlük kuralları

Süreçler

10 Roller, yetkiler ve durum makineleri
11 Veri hattı — OSM'den yol grafına
12 Optimizasyon akışı
13 Dinamik yeniden rotalama senaryoları

Algoritma

14 RouteWise algoritması
15 Amaç fonksiyonu — maliyet modeli
16 Algoritma karşılaştırması — iki seviye

Çıktı

17 Analiz, Excel raporları ve tasarruf yüzdesi
18 Geliştirme yol haritası
19 Maliyet karşılaştırması — matplotlib çıktısı

Yol Planı

20 Genel görünüm ve takvim
21 Faz 0–1a · hazırlık ve kod iskeleti
22 Faz 1b–1c · arayüz ve API
23 Faz 2–3 · kritik yol
24 Faz 4–7 · katmanlar ve kapanış
25 Riskler ve önlemler

Problem — tek depo, çok araç, kapasiteli dağıtım



Her araç depodan çıkar, kapasitesi kadar teslimat yapar, gün içinde depoya döner.

Kısıtlar ve değişkenler

BOYUT	TİP	MODELLEME
Araç kapasitesi	SERT	Σ paket ağırlığı ≤ kapasite · aşılamaz
Çalışma günü süresi	SERT	toplam rota süresi ≤ gün sınırı
Teslimat önceliği	YUMUŞAK	amaç fonksiyonunda ceza katsayısı
Toplam mesafe	AMAÇ	yol grafı + Dijkstra
Yakıt tüketimi	AMAÇ	araç bazlı L/100km × km
Sürücü maliyeti	AMAÇ	süre × saatlik ücret
Dağıtım merkezi	YAPISAL	**tek** depo

Neden sezgisel algoritma zorunlu — problem büyüklüğü

Tek araç için olası ziyaret sırası sayısı: (n−1)! / 2

DURAK	OLASI ROTA	ÖLÇEK
5	12	tam arama mümkün
8	2.520	tam arama mümkün
10	181.440	DFS-exhaustive sınırı
15	43,6 milyar	pratikte imkânsız
20	$6{,}1 \times 10^{16}$	pratikte imkânsız

Kapsam sınırı

KAPSAM İÇİ

- ✓ Bandırma içi dağıtım
- ✓ Tek dağıtım merkezi
- ✓ Araç kapasitesi (sert)
- ✓ Gün sınırı (sert)
- ✓ Dinamik yeniden rotalama

KAPSAM DIŞI

- ✗ Müşteri zaman penceresi
- ✗ Çok depo (MDVRP)
- ✗ Şehirlerarası taşıma
- ✗ SMS bildirimi
- ✗ Harici rota API'si

Somut başarı kriterleri — proje önerisinden

1 Teslimat noktalarına göre otomatik ve optimize rota oluşturulabilmesi

2 Teslimat bilgisi değiştiğinde rotanın yeniden oluşturulabilmesi

3 Araç kapasitesi ve önceliklerin planlamaya dahil edilebilmesi

4 Toplam mesafe ve tahmini yakıt tüketiminin hesaplanabilmesi

5 Optimize rota ile standart rota farkının ölçülebilmesi

6 Gerçekleştirilen dağıtımlara ait verilerin saklanabilmesi

7 Haftalık / aylık / yıllık tasarruf analizlerinin raporlanabilmesi

8 Farklı araç ve dağıtım senaryolarında çalıştırılabilmesi

9 Geleneksel planlamaya kıyasla ölçülebilir verimlilik artışı

Ölçüm yöntemi: sayfa 16–17 · çıktılar: Excel, PDF, grafik

Backend

Bileşen	Sürüm / Not
Java	21 (LTS)
Spring Boot	3.4+
Spring Web	REST API
Spring Security	JWT kimlik doğrulama
Spring Data JPA	ORM
Hibernate Spatial	PostGIS tipleri
Liquibase	Şema migration + seed
Spring Mail	E-posta bildirimi
Gradle	Derleme

Veritabanı

Bileşen	Sürüm / Not
PostgreSQL	17
PostGIS	3.6 — coğrafi tipler/sorgular
pgRouting	3.8 — yol grafı, referans Dijkstra

Veri hazırlığı

Araç	İş
OpenStreetMap	Ham yol verisi (ODbL)
Geofabrik	`turkey-latest.osm.pbf` ~619 MB
osmium-tool	Bandırma poligonuyla kırpma
osm2pgrouting	Grafı PostGIS'e yükleme

Python servisi

Bileşen	Sürüm / Not
Python	3.12
FastAPI	REST servis
uvicorn	ASGI sunucu
pydantic	İstek/yanıt doğrulama
numpy	Matris işlemleri
pandas	Agregasyon, zaman serisi
scipy	**Anlamlılık testleri**
matplotlib	Yayın kalitesi figür (SVG/PDF)
plotly	İnteraktif grafik
openpyxl	Excel dışa aktarma
pytest	Test

Frontend

Bileşen	Sürüm / Not
React	19
TypeScript	strict mod
Vite	Derleme / dev sunucu
MapLibre GL	Harita, rota çizimi
TanStack Query	Sunucu durumu
react-hook-form + zod	Form + doğrulama
Recharts	İnteraktif grafik

Altyapı

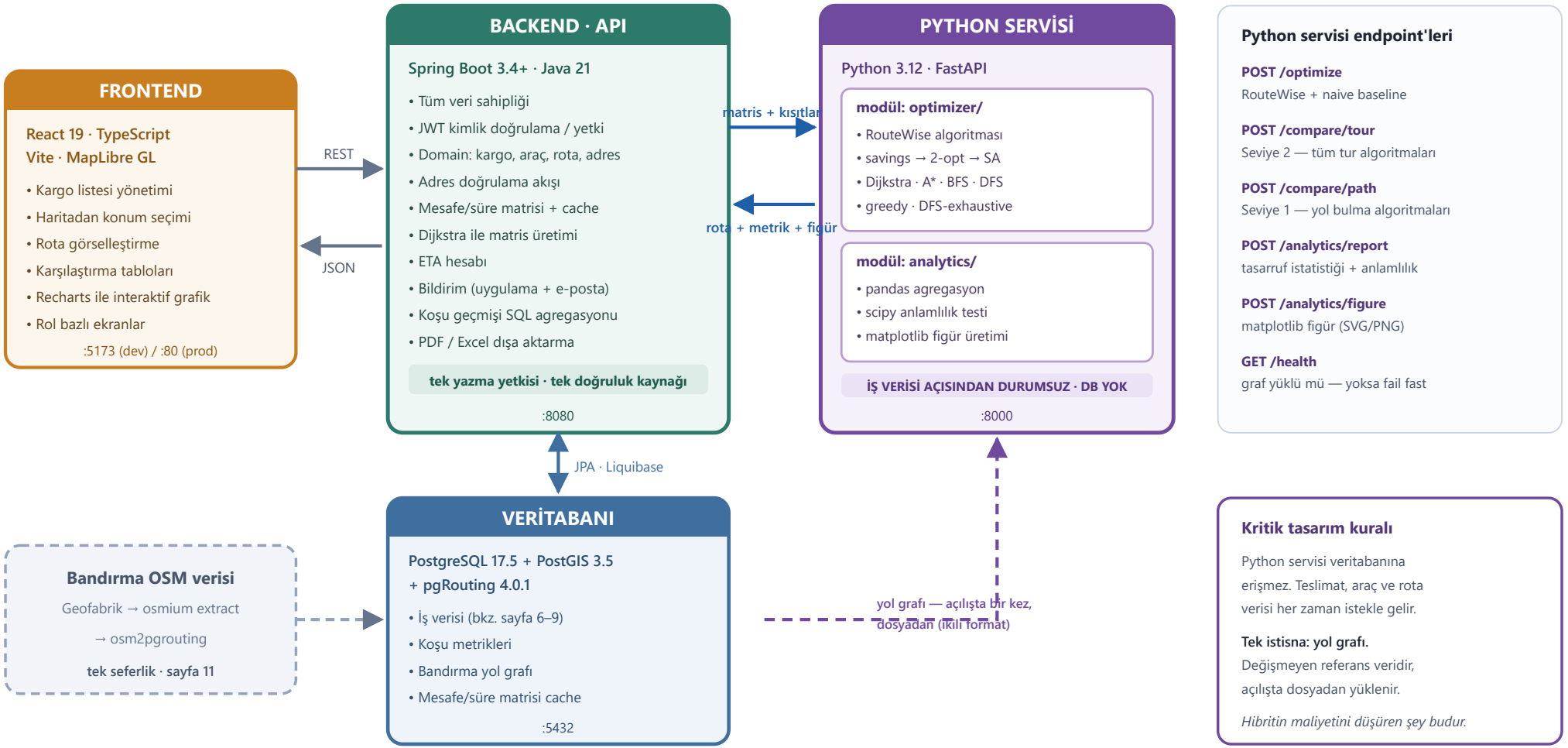
Bileşen	Not
Docker	Tüm servisler container'da
Docker Compose	Orkestrasyon
nginx	Frontend prod sunumu
Testcontainers	Gerçek DB ile entegrasyon testi

Algoritmalar — kendi implementasyonumuz

Algoritma	Rolü
Clarke-Wright savings	Başlangıç çözümü
2-opt	Rota içi yerel arama
relocate / exchange / Or-opt	Rotalar arası yerel arama
Simulated annealing	Metasezgisel
Dijkstra	Matris + referans optimum
A*	Karşılaştırma
BFS / DFS	Karşılaştırma
Nearest Neighbor	Karşılaştırma taban çizgisi

Hazır çözücü kullanılmıyor. OR-Tools veya benzeri bir VRP kütüphanesi yerine algoritma projeye özgü geliştirilir; özgünlük iddiasının kodda karşılığı budur.

Harici rota API'si yok. Google Maps / OpenRouteService kullanılmaz — mesafe ve süre matrisi kendi yol grafımızdan hesaplanır. Kota, API anahtarı ve internet bağımlılığı yoktur.



Python servisi endpoint'leri

POST /optimize
RouteWise + naive baseline

POST /compare/tour
Seviye 2 — tüm tur algoritmaları

POST /compare/path
Seviye 1 — yol bulma algoritmaları

POST /analytics/report
tasarruf istatistiği + anlamlılık

POST /analytics/figure
matplotlib figür (SVG/PNG)

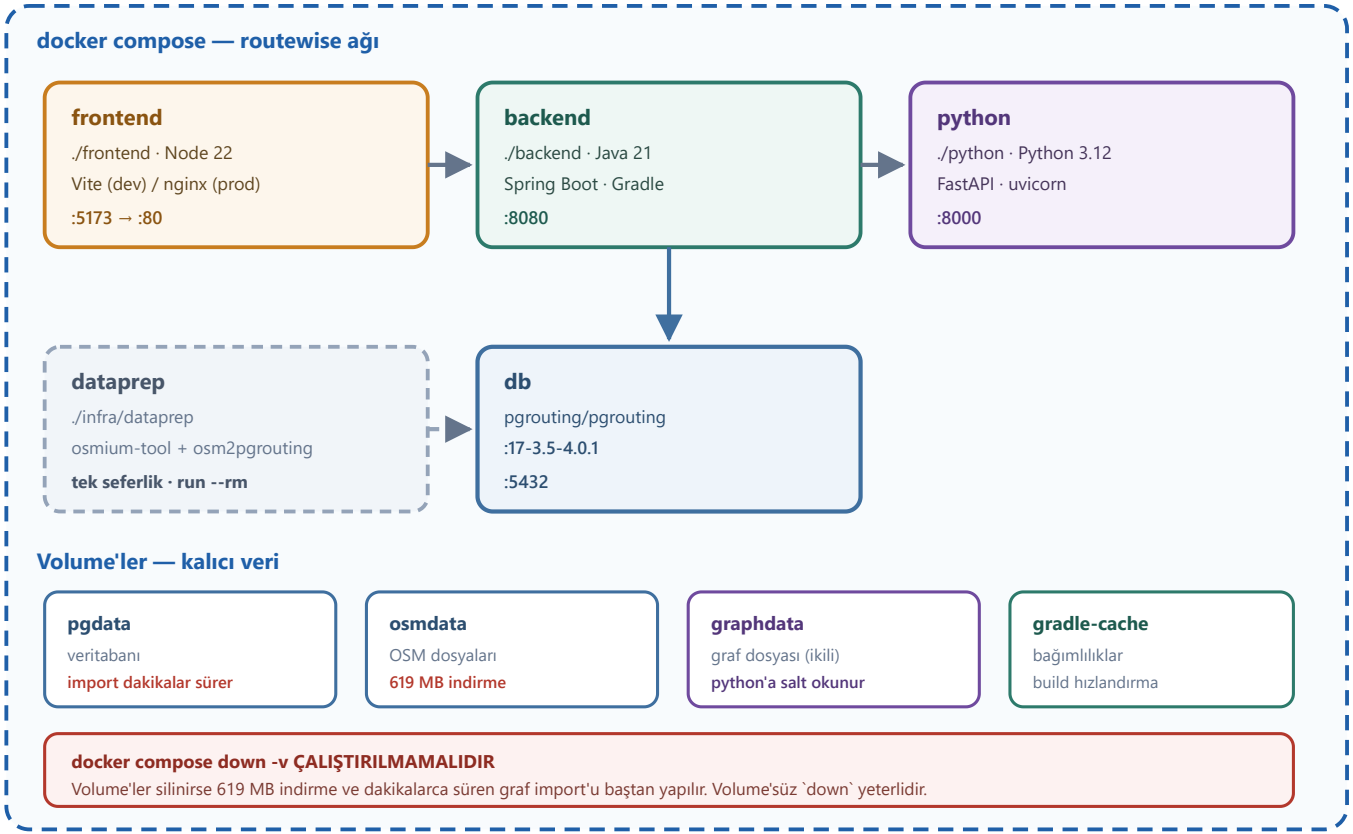
GET /health
graf yüklü mü — yoksa fail fast

Kritik tasarım kuralı

Python servisi veritabanına erişmez. Teslimat, araç ve rota verisi her zaman istekle gelir.

Tek istisna: yol grafı.
Değişmeyen referans veridir, açılışta dosyadan yüklenir.

Hibritin maliyetini düşüren şey budur.



Başlatma sırası ve healthcheck

db	— · pg_isready
backend	db healthy · /actuator/health
python	— · /health graf yoksa FAIL FAST
frontend	backend (gevşek) · HTTP 200

Python servisi graf dosyasını bulamazsa sessizce başlamaz — hatayı erken ve açıkça verir.

Veri hazırlığı — tek seferlik, 4 adım

```
compose run --rm dataprep download
compose run --rm dataprep clip
compose run --rm dataprep load
compose run --rm dataprep export
```

Adımlar ayrıdır: biri patlarsa baştan başlanmaz.

Windows'a özgü zorunlu ayarlar

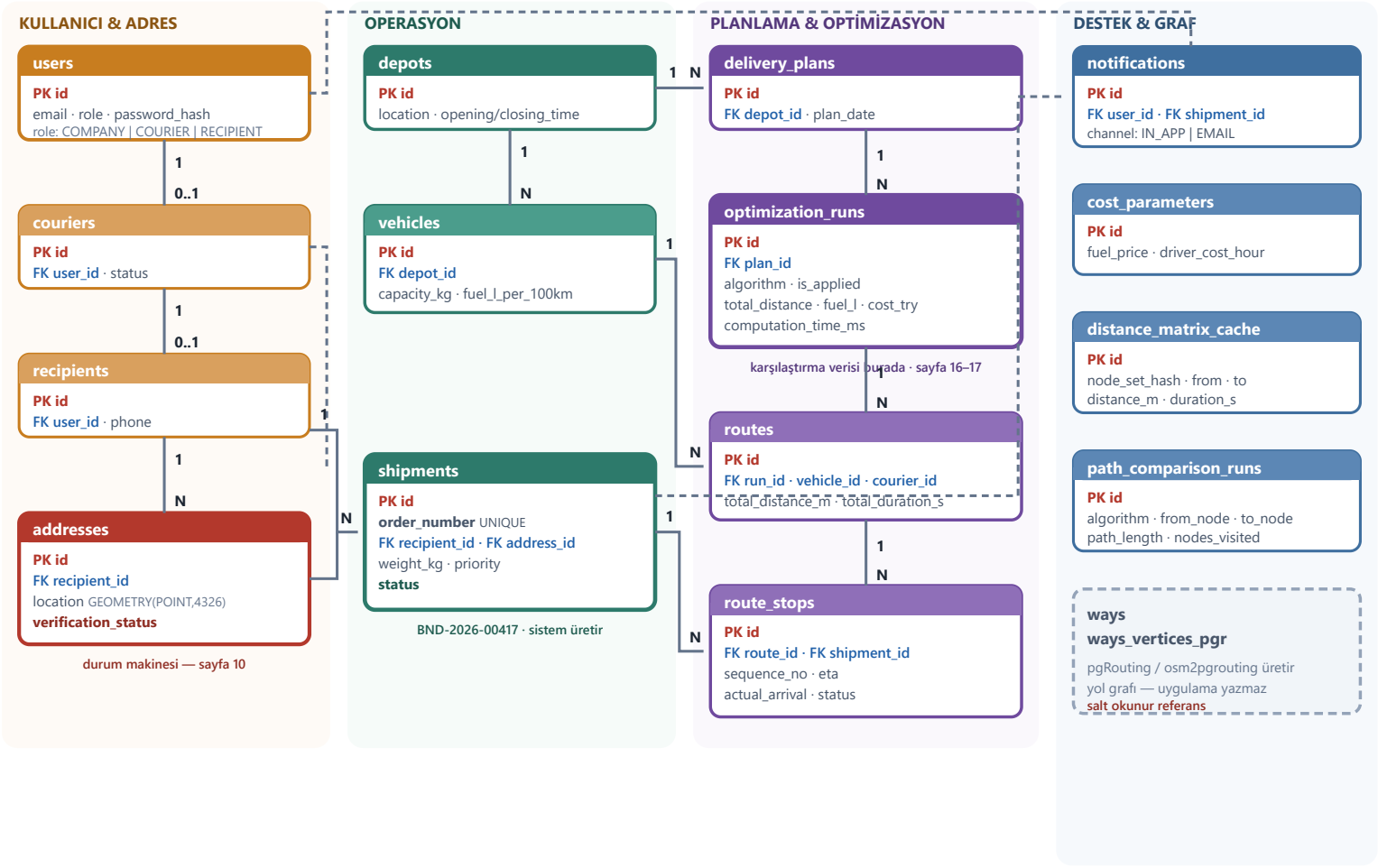
Vite hot reload tetiklenmez
bind-mount dosya olayları iletilmez → watch.usePolling: true · host: '0.0.0.0'

Spring Boot her değişimde imaj derlerse
iterasyon ölür → bootRun + devtools, kaynak bind-mount, .gradle named volume

Her restart'ta graf yeniden yüklenir
→ kompakt ikili format, JSON değil

Yapılandırma ve ortam

- Sırlar ve ortam ayarları **`.env`** dosyasında; **`.env.example`** repoda versiyonlanır, **`.env`** .gitignore'da
- Servisler birbirine **servis adıyla** erişir (db, backend, python) — IP veya localhost değil
- **postgis/postgis kullanılmıyor**: pgRouting içermiyor. Alternatif: nickblah/pgrouting (yama sürümüne kadar sabitlenebilir etiket)
- Geliştirme ve teslim **aynı compose** üzerinden — danışman/jüri tek komutla sistemi ayağa kaldırabilmeli



Gösterim

PK birincil anahtar

FK yabancı anahtar

1 — N bire-çok ilişki

1 — 0..1 isteğe bağlı bire-bir

Kesikli çizgi: zayıf / opsiyonel bağ

Kalın çerçeve: iş kuralı yoğun tablo

Tüm alanlar: sayfa 7–8

VARSAYIM — revize edilebilir

1. Alıcı **hesap sahibidir**; kargo bir alıcı kullanıcısına bağlanır. Hesapsız alıcı senaryosu modellenmedi.

2. Kurye–araç ataması **rota üzerinden** yapılır (günlük). Kalıcı araç sahipliği yok.

3. Bir kargo farklı günlerde yeniden planlanabilir → **shipments 1:N route_stops**.

4. Depo tablosu tek kayıt tutar ama çoklu depoya açıktır.

Özet

13 uygulama tablosu + 2 pgRouting tablosu

Coğrafi alan: addresses.location, depots.location

users

Alan	Tip	Kısıt
id	BIGSERIAL	PK
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL · BCrypt
full_name	VARCHAR(150)	NOT NULL
phone	VARCHAR(20)	
role	VARCHAR(20)	NOT NULL
is_active	BOOLEAN	DEFAULT true
created_at	TIMESTAMPTZ	NOT NULL
updated_at	TIMESTAMPTZ	

role: COMPANY · COURIER · RECIPIENT. Rol bazlı ek alanlar ayrı profil tablolarında tutulur — tek tabloda null alan yığılmaz.

couriers

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users · **UNIQUE**
employee_code	VARCHAR(30)	UNIQUE
status	VARCHAR(20)	NOT NULL
created_at	TIMESTAMPTZ	

status: AVAILABLE · ON_ROUTE · OFF_DUTY. `user_id` UNIQUE olduğu için ilişki **1:1**'dir.

recipients

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users · **UNIQUE**
default_address_id	BIGINT	FK→addresses · NULL
created_at	TIMESTAMPTZ	

addresses iş kuralı yoğun

Alan	Tip	Kısıt
id	BIGSERIAL	PK
recipient_id	BIGINT	FK→recipients, NOT NULL
address_text	VARCHAR(500)	NOT NULL
district	VARCHAR(100)	
neighbourhood	VARCHAR(100)	
location	GEOMETRY (POINT,4326)	NOT NULL · **GIST index**
snapped_node_id	BIGINT	→ ways_vertices_pgr
snap_distance_m	NUMERIC(8,2)	yola uzaklık
verification_status	VARCHAR(20)	NOT NULL
verified_by	BIGINT	FK→users · NULL
verified_at	TIMESTAMPTZ	NULL
rejection_reason	VARCHAR(300)	NULL
change_count_today	SMALLINT	DEFAULT 0 · limit
created_at / updated_at	TIMESTAMPTZ	

verification_status: DRAFT · PENDING · VERIFIED · REJECTED. **Yalnızca VERIFIED adresler optimizasyona girer.** Alıcı adresi değiştirdiğinde durum otomatik PENDING'e düşer — aksi hâlde firma doğruladığını sanırken başka adrese rota çizilir.

snapped_node_id çifte iş görür: hem yol grafına bağlanma noktasıdır (rota hesabı için zorunlu), hem de yanlış işaretleme kontrolüdür — `snap_distance_m` eşiği aşarsa nokta reddedilir.

depots

Alan	Tip	Kısıt
id	BIGSERIAL	PK
name	VARCHAR(120)	NOT NULL
address_text	VARCHAR(500)	
location	GEOMETRY (POINT,4326)	NOT NULL · GIST
snapped_node_id	BIGINT	→ ways_vertices_pgr
opening_time	TIME	çıkış saati
closing_time	TIME	dönüş saati
max_route_duration_min	INTEGER	NOT NULL

max_route_duration_min projenin ikinci sert kısıtıdır — "tüm teslimatlar gün içinde bitsin" kuralının veri karşılığı. Rota süresi bu değeri aşamaz.

vehicles

Alan	Tip	Kısıt
id	BIGSERIAL	PK
depot_id	BIGINT	FK→depots, NOT NULL
plate	VARCHAR(20)	UNIQUE, NOT NULL
capacity_kg	NUMERIC(8,2)	NOT NULL · sert kısıt
capacity_volume_m3	NUMERIC(8,2)	
fuel_l_per_100km	NUMERIC(5,2)	NOT NULL · maliyet
fuel_type	VARCHAR(20)	DIESEL · GASOLINE
fixed_cost_per_day	NUMERIC(10,2)	opsiyonel bileşen
is_active	BOOLEAN	DEFAULT true

Filo şimdilik sabittir — Liquibase seed verisiyle yüklenir, yönetim ekranı yazılmaz. Ancak koda gömülmez: tablo baştan vardır, böylece "farklı araç senaryolarında çalıştırılabilmesi" kriteri sonradan karşılanabilir.

shipments kargo

Alan	Tip	Kısıt
id	BIGSERIAL	PK
order_number	VARCHAR(20)	UNIQUE, NOT NULL
recipient_id	BIGINT	FK→recipients
address_id	BIGINT	FK→addresses
weight_kg	NUMERIC(8,2)	NOT NULL · kapasite
volume_m3	NUMERIC(8,2)	
priority	SMALLINT	DEFAULT 0 · ceza
service_time_min	INTEGER	teslim süresi
status	VARCHAR(20)	NOT NULL
postponed_to	DATE	NULL · erteleme
created_at / updated_at	TIMESTAMPTZ	

order_number sistem tarafından üretilir (`BND-2026-00417`) — elle giriş yoktur, yanlış yazma riski sıfırdır.
status: BEKLEMEDE · PLANLANDI · YOLDA · TESLİM_EDİLDİ · ERTELENDİ.

delivery_plans günlük plan

Alan	Tip	Kısıt
id	BIGSERIAL	PK
depot_id	BIGINT	FK→depots
plan_date	DATE	NOT NULL
status	VARCHAR(20)	TASLAK·SEVK·TAMAM
created_by	BIGINT	FK→users
created_at	TIMESTAMPTZ	
UNIQUE (depot_id, plan_date) — gün başına tek plan		

optimization_runs her algoritma koşusu

Alan	Tip	Kısıt
id	BIGSERIAL	PK
plan_id	BIGINT	FK→delivery_plans
algorithm	VARCHAR(40)	NOT NULL
is_applied	BOOLEAN	uygulanan rota mı
is_baseline	BOOLEAN	naive referans mı
total_distance_m	NUMERIC(12,2)	
total_duration_s	INTEGER	
total_fuel_l	NUMERIC(10,3)	
total_cost_try	NUMERIC(12,2)	tasarruf hesabı
vehicle_count	SMALLINT	
computation_time_ms	INTEGER	hız karşılaştırması
random_seed	BIGINT	SA tekrarlanabilirlik
created_at	TIMESTAMPTZ	

Projenin ölçüm omurgası bu tablodur. Her koşuda RouteWise *ve* naive baseline kaydedilir; tasarruf yüzdesi ikisinin `total_cost_try` farkından türetilir. Karşılaştırma koşusunda aynı `plan_id` altında her algoritma için bir satır oluşur.

algorithm: NAIVE · GREEDY_NN · SAVINGS · ROUTEWISE · DFS_EXHAUSTIVE.
random_seed olmadan SA aynı girdiye farklı sonuç verir ve sonuçlar tekrar üretilemez — tez için zorunlu alan.

routes araç başına rota

Alan	Tip	Kısıt
id	BIGSERIAL	PK
run_id	BIGINT	FK→optimization_runs
vehicle_id	BIGINT	FK→vehicles
courier_id	BIGINT	FK→couriers · NULL
total_distance_m	NUMERIC(12,2)	
total_duration_s	INTEGER	≤ max_route_duration
total_load_kg	NUMERIC(8,2)	≤ capacity_kg
status	VARCHAR(20)	
started_at / completed_at	TIMESTAMPTZ	NULL

route_stops duraklar

Alan	Tip	Kısıt
id	BIGSERIAL	PK
route_id	BIGINT	FK→routes
shipment_id	BIGINT	FK→shipments
sequence_no	INTEGER	NOT NULL · ziyaret sırası
distance_from_prev_m	NUMERIC(10,2)	
duration_from_prev_s	INTEGER	
eta	TIMESTAMPTZ	tahmini varış
actual_arrival	TIMESTAMPTZ	NULL
status	VARCHAR(20)	
UNIQUE (route_id, sequence_no)		

eta alanı alıcıya gönderilen bildirimin kaynağıdır (sayfa 12). `actual_arrival` ile farkı, tahmin doğruluğunun ölçülmesini sağlar — raporda ek bir metrik olarak kullanılabilir.

notifications

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users
shipment_id	BIGINT	FK→shipments · NULL
type	VARCHAR(30)	ETA·ERTELEME·DOĞRULAMA
channel	VARCHAR(20)	IN_APP · EMAIL
payload	JSONB	mesaj içeriği
sent_at	TIMESTAMPTZ	
read_at	TIMESTAMPTZ	NULL

SMS kapsam dışıdır; `channel` alanı ileride genişletilebilir şekilde tasarlanmıştır.

cost_parameters

Alan	Tip	Kısıt
id	BIGSERIAL	PK
fuel_price_per_l	NUMERIC(8,2)	NOT NULL
driver_cost_per_hour	NUMERIC(8,2)	NOT NULL
priority_penalty	NUMERIC(8,2)	ceza katsayısı
effective_from	DATE	NOT NULL

Parametreler **tarihli** tutulur: yakıt fiyatı değiştiğinde geçmiş koşuların maliyeti bozulmaz. Her optimizasyon çağrısında o günün parametreleri Python servisine gönderilir — kod değişmeden sonuçlar değişir.

distance_matrix_cache

Alan	Tip	Kısıt
id	BIGSERIAL	PK
node_set_hash	CHAR(64)	SHA-256 · INDEX
from_node_id	BIGINT	NOT NULL
to_node_id	BIGINT	NOT NULL
distance_m	NUMERIC(10,2)	Dijkstra sonucu
duration_s	INTEGER	maxspeed'den
computed_at	TIMESTAMPTZ	
UNIQUE (node_set_hash, from_node_id, to_node_id)		

Bu cache olmadan sistem kullanılamaz. Metasezgisel algoritma matrisi koşu başına bir kez ister, ama algoritma ayarı sırasında yüzlerce koşu yapılır. Aynı durak kümesi → aynı hash → tekrar hesap yok.

path_comparison_runs

Alan	Tip	Kısıt
id	BIGSERIAL	PK
algorithm	VARCHAR(20)	DIJKSTRA·ASTAR·BFS·DFS
from_node_id / to_node_id	BIGINT	NOT NULL
path_length_m	NUMERIC(10,2)	
computation_time_ms	NUMERIC(10,3)	
nodes_visited	INTEGER	
deviation_pct	NUMERIC(6,2)	Dijkstra'ya göre
created_at	TIMESTAMPTZ	

Seviye 1 (yol bulma) deneylerinin kaydı. Seviye 2 (tur sırası) kayıtları `optimization_runs` tablosundadır. İki seviye ayrı tutulur çünkü ölçülen metrikler farklıdır — bkz. sayfa 16.

Tüm ilişkiler ve kardinaliteler

İlişki	Kard.	Açıklama
users → couriers	**1 : 0..1**	profil
users → recipients	**1 : 0..1**	profil
users → notifications	**1 : N**	
recipients → addresses	**1 : N**	çoklu adres
recipients → shipments	**1 : N**	
addresses → shipments	**1 : N**	aynı adrese çok kargo
depots → vehicles	**1 : N**	
depots → delivery_plans	**1 : N**	gün başına bir plan
delivery_plans → optimization_runs	**1 : N**	her algoritma bir koşu
optimization_runs → routes	**1 : N**	her araç bir rota
routes → route_stops	**1 : N**	sıralı duraklar
shipments → route_stops	**1 : N**	farklı koşu/günde tekrar
vehicles → routes	**1 : N**	
couriers → routes	**1 : N**	günlük atama
shipments → notifications	**1 : N**	

Bütünlük kuralları

1. Yalnızca `verification_status = VERIFIED` adresli kargo plana dahil edilir.
2. Bir plan altında en fazla **bir** koşu `is_applied = true` olabilir.
3. `routes.total_load_kg ≤ vehicles.capacity_kg` — uygulama ve algoritma seviyesinde zorlanır.
4. `routes.total_duration_s ≤ depots.max_route_duration_min × 60`
5. Alıcı adresi değiştirince `verification_status → PENDING` (tetikleyici / servis katmanı).
6. Kargo `YOLDA` iken adres **değiştirilemez**.
7. Silme davranışı: profil ve rota zincirinde `ON DELETE CASCADE`; kargo ve koşu kayıtlarında `ON DELETE RESTRICT` — geçmiş ölçüm verisi korunur.

10 Roller, Yetkiler ve Durum Makineleri

Yetki matrisi

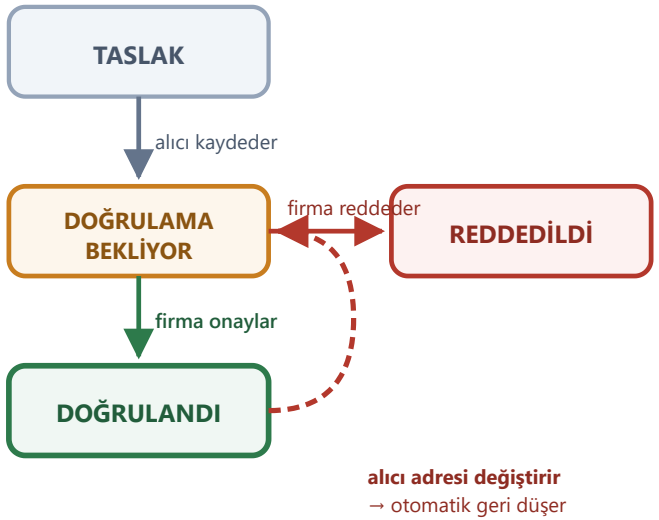
Yetki	Firma	Kurye	Alıcı
Kargo kaydı oluşturma	✓	—	—
Sipariş no üretimi	otomatik	—	—
Adres doğrulama / reddetme	✓	—	—
Rota oluşturma, optimizasyon	✓	—	—
Rotayı görme	tümü	kendi	kendi teslimatı
Teslimat durumu güncelleme	✓	kendi durakları	—
Kendi adresini değiştirme	✓	—	✓
Teslimatı erteleme	✓	—	✓
Araç / filo yönetimi	seed	—	—
Karşılaştırma koşusu başlatma	✓	—	—
Tasarruf raporları, Excel/PDF	✓	—	—

Kimlik doğrulama **JWT** ile; her endpoint rol bazlı korunur. Test stratejisinde her endpoint için **rol sızıntısı testi** zorunludur — alıcı başkasının teslimatını görememelidir.

Alıcı adresi doğrudan değiştirebilir — güçlü bir özelliktir ama tasarlanmazsa sistemi kırar. Zorunlu önlemler:

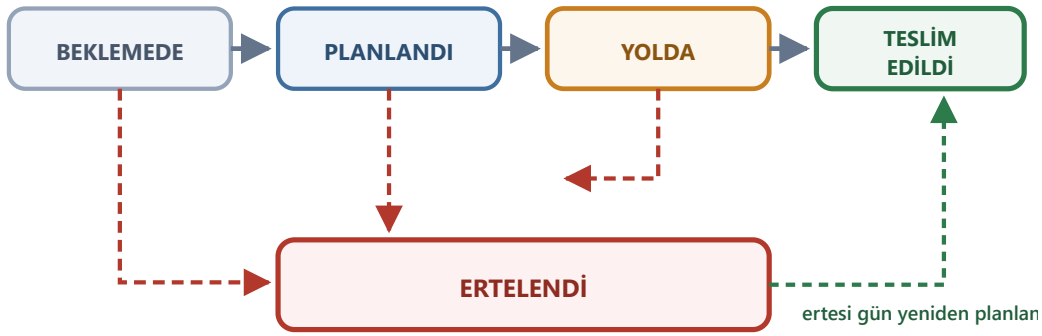
- Yeni adres rotayı süre sınırının üstüne çıkarırsa teslimat **otomatik ERTELENDİ'ye** düşer, alıcı bilgilendirilir
- Teslimat **YOLDA** olunca adres **kilitlenir**
- Maliyet artışı eşiği aşarsa sistem erteleme önerir
- Teslimat başına **günlük değişiklik limiti**
- Kuryeye **açık bildirim**; eski durak listesinden kaldırılır

Adres doğrulama durum makinesi



Yalnızca DOĞRULANDI adresler optimizasyona girer.
Bu kural olmadan firma doğruladığını sanırken başka adrese rota çizilir.

Teslimat durum makinesi



ERTELENDİ'ye geçiş sebepleri

- Alıcı "başka gün gelsin" der
- Teslimat denemesi başarısız
- Adres değişikliği rotayı süre sınırının üstüne çıkarır → **otomatik** durak kümeden çıkar, rota yeniden hesaplanır

Veri Hattı — OSM'den Yol Grafına

Tek seferlik kurulum · harici API yok

KURULUM HATTI — bir kez çalışır

1 download
Geofabrik
turkey-latest.osm.pbf
619 MB
08.10.2026 verisi

→

2 clip
osmium-tool
osmium extract --polygon
bandirma.poly
→ birkaç on MB

→

3 load
osm2pgrouting 2.3.8
→ ways
→ ways_vertices_pgr
topolojiyi kendi kurar

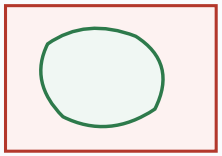
→

4 export
graf → ikili dosya
graphdata volume'üne
Python açılışta okur
JSON değil — restart hızı

Neden kendimiz kırpıyoruz

Geofabrik Türkiye için
alt bölge yayınlamıyor:
"No sub regions are defined for this region."
Tek dosya var.

bbox değil, poligon



dikdörtgen vs ilçe sınırı

Bbox dikdörtgeni komşu ilçelerden gereksiz yol ağı taşır: graf şişer, Dijkstra yavaşlar.

Aynı poligon geofence kontrolünde de kullanılır — bir iş, iki fayda.

ÇALIŞMA ZAMANI — matris hesabı

Durak listesi
depo + teslimat noktaları

→

Kendi Dijkstra'mız
graf üzerinde, her çift için

→

Mesafe + süre matrisi
N × N
süre = uzunluk / maxspeed

↔

distance_matrix_cache
node_set_hash ile aranır
aynı küme → tekrar hesap yok

→

Python
optimizer

Harici rota API'si neden gerekmiyor

Hazır API ile (OpenRouteService / Google)	Kendi grafımızla
Günlük istek kotası	sınırsız
Matris başına ≈59 konum sınırı	sınır yok
API anahtarı yönetimi	gerekmez
Demo gününde internet riski	tamamen yerel
Ham graf alınamaz	graf elimizde

Graf olmadan Seviye 1 (Dijkstra/A*/BFS/DFS) karşılaştırması zaten yapılamazdı.

Doğruluk referansı

pgRouting'in kendi `pgr_dijkstra` fonksiyonu, bizim Dijkstra implementasyonumuzun doğruluk referansıdır: **aynı girdi → aynı mesafe.**

Test stratejisinin zorunlu maddesidir (sayfa 18).

osm2pgrouting — bilinen kısıt

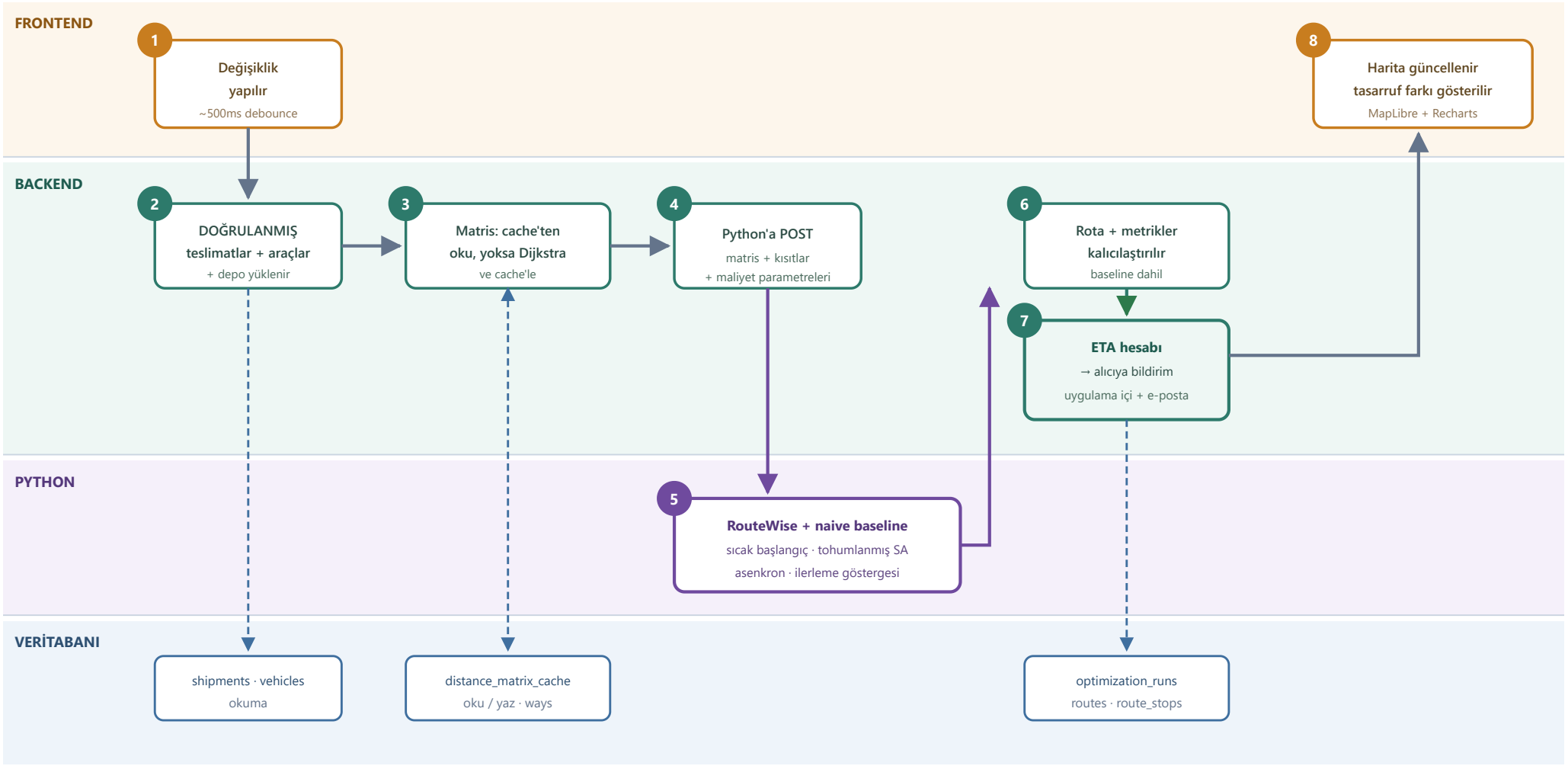
Araç veriyi **tamamen belleğe yükler**; bu, büyük ağlarda ciddi bir sınırdır.

Bandırma ölçeğinde önemsizdir — kapsam daraltmanın üçüncü somut getirisi.

Araç bakımlıdır: Debian'da 3.0.0 sürümü Mayıs 2026'da testing'e geçmiştir.

TETİKLEYİCİLER — her biri anında yeniden hesaplama başlatır

- Paket eklendi / çıkarıldı
- Alıcı adresi değiştirdi
- Alıcı "başka gün gelsin" dedi
- Firma adresi doğruladı
- Teslimat tamamlandı (dağıtım sırasında)



İlke

Dağıtıma çıkmadan önce tam optimizasyon koşar ve en tasarruflu rota seçilir. Dağıtım sırasında bir olay olursa: **teslim edilmiş duraklar sabitlenir**, kalan duraklar yeniden optimize edilir ve kuryeye bildirilir.

Canlı GPS gerekmiyor

Yeniden optimizasyon **son tamamlanan duraktan** başlar, canlı konumdan değil.

Kurye her teslimatı tamamlandı işaretlediği için sistem nerede olduğunu zaten bilir.

SENARYO A — Alıcı "başka gün gelsin" der



4 durak · 18,4 km

3 durak · 14,1 km · −23%

Kargo **ERTELENDİ** durumuna geçer, `postponed_to` alanı dolar.

Rota yeniden hesaplanır, kuryeye bildirilir. Kargo ertesi gün yeniden planlanır.

Risk düşüktür: durak yalnızca kümeden çıkar, yeni kısıt gelmez.

SENARYO B — Alıcı adresini değiştirir



Adres **DOĞRULAMA_BEKLİYOR**'a düşer; firma onaylayana kadar rotaya girmez.

Onaylanınca kalan duraklar yeniden optimize edilir.

Risk yüksektir: yeni konum rotayı olurluluk sınırının dışına çıkarabilir.

Bu yüzden aşağıdaki beş önlem zorunludur.

Zorunlu güvenlik önlemleri — tasarlanmazsa sistem kırılır

Risk	Önlem
Yeni adres rotayı maksimum süre sınırının üstüne çıkarır	Teslimat **otomatik ERTELENDİ**'ye düşer, alıcı bilgilendirilir
Kurye o durağa giderken adres değişir	Teslimat **YOLDA** olunca adres kilitlenir, değişiklik kabul edilmez
Kurye o bölgeden çoktan geçmiş, yeni nokta çok geride	Maliyet artışı **eşiği aşarsa** sistem erteleme önerir
Tekrarlayan değişiklikle rota titreşimi	Teslimat başına **günlük değişiklik limiti** (change_count_today)
Kurye sessizce eski adrese gider	Kuryeye **açık bildirim**; eski durak listesinden kaldırılır

14 RouteWise Algoritması

Hazır çözücü yok · projeye özgü implementasyon

GİRDİ

- Mesafe matrisi (N×N)
- Süre matrisi (N×N)
- Teslimatlar: ağırlık, öncelik
- Araçlar: kapasite, tüketim
- Depo: max_route_duration
- Maliyet parametreleri
- **random_seed**

1 Clarke-Wright savings

Başlangıç çözümü.

Her teslimat tek başına bir rotayken başlanır; birleştirmenin kazancı

$s(i,j) = d(0,i) + d(0,j) − d(i,j)$

en yüksek olan çiftler sırayla birleştirilir.

Kapasite ve süre farkında: olursuz birleştirme atlanır.

2 Yerel arama

2-opt	rota içi — iki kenarı değiştirir
relocate	durağı başka rotaya taşır
exchange	iki durağı takas eder
Or-opt	2–3 duraklık dizi taşır

her hamlede olurluluk kontrol edilir

3 Simulated annealing

Kötüleştiren hamleler de olasılıkla kabul edilir → yerel optimumdan kaçış.

$P = \exp(-\Delta / T)$, T azalır

Tohumlanmış (seeded): tohumsuz aynı girdi farklı rota verir, harita sebepsiz zıplar, sonuç tekrar üretilemez.

Olurluluk kapısı — her hamlede

Her yerel arama hamlesi ve her SA adımı, kabul edilmeden önce iki sert kısıta karşı sınanır:

Σ ağırlık ≤ araç kapasitesi	vehicles.capacity_kg
toplam süre ≤ gün sınırı	depots.max_route_duration_min

Bu kontroller olmadan algoritma **sessizce geçersiz rota döndürür** ve kimse fark etmez — test stratejisindeki en kritik test sınıfı budur.

Sıcak başlangıç (warm start)

Kargo listesi her değiştiğinde SA **sıfırdan değil, önceki rotadan** başlar.

İki faydası vardır:

- **Hız** — iyi bir çözümden başlanır, daha az iterasyon gerekir
- **Kararlılık** — tek paket eklenince rota baştan aşağı değişmez; kullanıcı sistemin ne yaptığını anlar ve güvenir

Tohumlanmış rastgelelik + sıcak başlangıç birlikte "anında tetikleme"yi kullanılabilir kılar.

Neden savings ve 2-opt kullanılabiliyor

Çalışmanın erken aşamasında kapsam **zaman penceresi (VRPTW)** içeriyordu. O kapsamda bu iki yöntem **kullanılamazdı:**

- **Savings** tek depolu, penceresiz CVRP için tasarlanmıştır; birleştirme adımında pencere olurluluğunu değerlendiremez.
- **2-opt** bir rota segmentini **ters çevirir**. Ters çevirme segment boyunca tüm varış saatlerini kaydırır ve pencereleri ihlal eder.

Kapsam gün sınırına daraltılınca gerekçe ortadan kalktı:

simetrik matriste ters çevirme mesafeyi azaltıyorsa süreyi de azaltır, dolayısıyla süre kısıtı **basit bir toplam kontrolüyle** korunur.

Sonuç: literatürde en iyi dokümante edilmiş klasik kurulum kullanılabiliyor.

Problem sınıfı ve beklenti

CVRP **NP-zordur**. Durak sayısı arttıkça olası rota sayısı faktöriyel büyür; gerçek optimumu bulmak küçük örnekler dışında mümkün değildir.

Bu yüzden hedef "optimal çözüm" değil, ölçülebilir şekilde iyi çözümdür.

Kalite iki referansa karşı ölçülür:

- **Naive baseline** — insan dispatcher davranışı → tasarruf %
- **DFS-exhaustive** (n ≤ 10) — gerçek optimum → sapma %

İkincisi raporun en güçlü nicel iddiasını üretir:

"RouteWise, gerçek optimumun %X içinde."

Maliyet, rota hesaplandıktan SONRA eklenen bir rapor alanı değildir.

Amaç fonksiyonunun parçasıdır — algoritmanın her kararını bu değerler yönlendirir. "Sadece en kısa yol değil, değişkenler birlikte" cümlesinin kodda karşılığı burasıdır.

Amaç fonksiyonu

$$\min Z = \Sigma_r \left(\text{mesafe}(r) \cdot c_{km} + \text{yakıt}(r) \cdot c_{yakıt} + \text{süre}(r) \cdot c_{sürücü} \right) + \Sigma_i \text{öncelik}(i) \cdot p$$

r: rotalar · i: teslimatlar · c: birim maliyetler (cost_parameters) · p: öncelik ceza katsayısı

SERT KISITLAR

Aşılamaz. İhlal eden çözüm cezalandırılmaz, **reddedilir.**

Araç kapasitesi

Σ shipments.weight_kg ≤ vehicles.capacity_kg

araç tanımından gelir

Maksimum rota süresi

routes.total_duration_s ≤ max_route_duration_min × 60

"gün içinde bitsin" kuralı

AMAÇ FONKSİYONU BİLEŞENLERİ — minimize edilir

Mesafe maliyeti

toplam mesafe (km)

kaynak: yol grafı + Dijkstra

→ optimization_runs.total_distance_m

Yakıt maliyeti

(L/100km × km) × birim fiyat

tüketim araç bazlıdır

→ total_fuel_l · cost_parameters

Sürücü maliyeti

süre (saat) × saatlik ücret

servis süresi dahil

→ driver_cost_per_hour

Süre maliyeti

seyahat + servis süresi

yol grafındaki maxspeed'den

→ total_duration_s

Öncelik ihlali cezası

shipments.priority × katsayı

yumuşak kısıt — reddedilmez, pahalılaşır

Araç sabit maliyeti

fixed_cost_per_day

az araçla çözmeyi teşvik eder

opsiyonel bileşen

Parametreler nereden gelir

cost_parameters — DB · tarihli kayıt → her çağrıda → **Python servisi** — istekle alır

Yakıt fiyatı veya sürücü maliyeti değiştiğinde kod değişmez; optimizasyon sonuçları kendiliğinden değişir.

Neden "sadece en kısa yol" yetmez

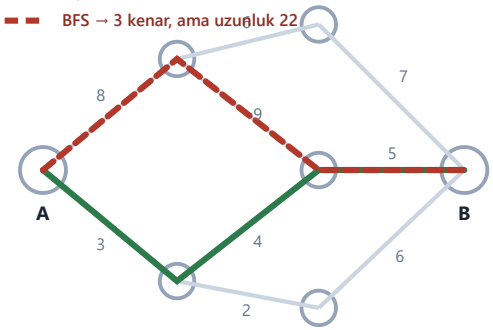
Aynı mesafedeki iki rota eşit değildir:

- Biri dar sokaklarda yavaş ilerler → **sürücü maliyeti** artar
- Diğeri daha uzun ama hızlı güzergâhtan gider → **süre ve yakıt** düşer
- Üçüncüsü yüksek öncelikli paketi güne çok geç bırakır → **ceza**

Yalnızca mesafeyi minimize eden algoritma bu üç farkı göremez.
RouteWise'ın amaç fonksiyonu görür.

SEVİYE 1 — YOL BULMA

"A noktasından B noktasına hangi yoldan gidilir?" · Bandırma yol grafı üzerinde · kayıt: path_comparison_runs



Dijkstra

Ağırlıklı grafta en kısa yolu **garanti eder**

REFERANS OPTİMUM

A*

Dijkstra ile **aynı yolu** daha az düğüm gezerek bulur

haversine sezgiseli

BFS

Kenar ağırlıklarını **YOK SAYAR**

en az kavşak ≠ en kısa

DFS

Optimallik garantisi yok

keşif sırasına bağlı

Rapor için değerli bulgu

Yol ağı **ağırlıklı** bir graftır; ağırlıkları yok sayan bir arama algoritması doğru cevabı veremez. BFS'in bu yanlışlığını sayısallaştırmak, algoritma seçiminin neden kritik olduğunu gösterir.

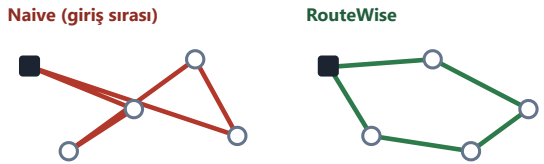
Ölçülen metrikler

yol uzunluğu (m) · hesaplama süresi (ms) · gezilen düğüm sayısı · Dijkstra optimumundan sapma (%)

Tümü `path_comparison_runs` tablosuna yazılır.

SEVİYE 2 — TUR SIRASI

"Hangi teslimat hangi sırayla, hangi araçla?" · mesafe matrisi üzerinde · kayıt: optimization_runs



Aynı duraklar, farklı sıra → farklı mesafe, süre, maliyet

Her koşuda naive baseline de hesaplanır ve kaydedilir: **tasarruf ölçümü bu karşılaştırmadan türetilir.**

Naive — giriş sırası

Taban çizgi: insan dispatcher davranışı. Optimizasyon yok.

Nearest Neighbor (greedy)

Açgözlü kurucu heuristik.
Çok hızlı, kalite orta.

Clarke-Wright savings

Klasik kurucu heuristik.
RouteWise'ın 1. adımı.

RouteWise — projenin kendi algoritması

savings (başlangıç) → 2-opt · relocate · exchange · Or-opt (yerel arama)
→ simulated annealing (tohumlanmış, sıcak başlangıçlı)

Her hamlede kapasite ve maksimum rota süresi olurluluğu kontrol edilir.

DFS-exhaustive

yalnızca n ≤ 10
Tüm permütasyonlar gezilir

GERÇEK OPTİMUM

Yöntem notu Hız karşılaştırması ancak implementasyonlar aynı dilde, benzer özenle, aynı veri yapıları üzerinde yazıldığında adildir. Her algoritma N kez koşturulur, medyan raporlanır.

Asıl ilginç sonuç tek başına hız değil, **kalite–süre dengesidir** — greedy hızlı ama kötü, DFS-exhaustive optimal ama kullanılamaz; RouteWise ikisinin arasındadır.

Analiz, Excel Raporları ve Tasarruf Yüzdesi

Projenin ana amacının ölçüldüğü yer

Tasarruf yüzdesi — ana metrik

$$\text{tasarruf \%} = (\ \text{cost}_{\text{naive}} - \text{cost}_{\text{routewise}}\ ) / \text{cost}_{\text{naive}} \times 100$$

her ikisi de optimization_runs tablosunda, aynı plan_id altında

Her optimizasyon koşusunda naive baseline **de** hesaplanıp kaydedildiği için tasarruf geçmişe dönük agrege edilebilir.

Veri akışı — durumsuzluk korunur

Spring Boot — SQL agregasyon → **Python analytics** — pandas · scipy · matplotlib → **React** — Recharts · indirme

Excel raporu openpyxl

Ham sayılar — danışmanın kendi analizini yapabileceği tablo.

- **Sayfa 1 — Özet**
  dönem · toplam km · yakıt L · maliyet TL · tasarruf %
- **Sayfa 2 — Koşu detayı**
  her optimization_run satır satır · algoritma · metrikler
- **Sayfa 3 — Algoritma karşılaştırma**
  Seviye 1 ve Seviye 2 tabloları · sapma % · süre ms
- **Sayfa 4 — Zaman serisi**
  haftalık / aylık / yıllık kırılım · pandas resample
- **Sayfa 5 — İstatistiksel test**
  Wilcoxon / t-test sonuçları · p değerleri · n

CSV dışa aktarma da aynı veriden türetilir.

PDF raporu matplotlib

Figürlerle birlikte sunuma ve teze hazır rapor.

- Kapak: dönem, depo, araç sayısı
- Tasarruf özeti — büyük punto, tek sayı
- Zaman serisi grafiği (haftalık/aylık/yıllık)
- Algoritma karşılaştırma tablosu
- Kalite–süre dengesi grafiği
- İstatistiksel anlamlılık sonucu
- Örnek rota haritası (önce / sonra)

Vektörel çıktı (SVG/PDF)
Tez raporuna doğrudan gömülür; ekran görüntüsü alınmaz, baskıda bozulmaz.

Görsel karşılaştırmalar

Grafik	Gösterdiği
Çubuk — algoritma × maliyet	hangisi ucuz
Çizgi — zaman × tasarruf %	iyileşme eğilimi
Dağılım — süre × sapma	kalite–hız dengesi
Yığılmış çubuk — maliyet kırılımı	yakıt / sürücü payı
Kutu grafiği — senaryo dağılımı	tutarlılık
Harita — önce / sonra rota	görsel kanıt

İki yerde, iki amaçla
React + Recharts → ekranda interaktif: hover, filtre
Python + matplotlib → raporda yayın kalitesi

İstatistiksel anlamlılık — iddiayı savunulabilir kılan adım

Tek başına "*RouteWise greedy'den %13 daha iyi*" zayıf bir iddiadır.
Jüri sorar: "Bu fark şans eseri olamaz mı? Kaç senaryoda denedin?"

Aynı senaryo kümesi tüm algoritmalarla koşturulur ve **eşleştirilmiş Wilcoxon işaretli sıra testi** uygulanır (dağılım varsayımı gerektirmez; alternatif paired t-test).

"RouteWise, greedy'ye göre anlamlı derecede daha iyi ($p < 0.001$, $n = 50$ senaryo)."

Maliyeti birkaç satır `scipy.stats` kodudur; kazancı, iddianın "benim sayılarım daha güzel"den savunulabilir bir bulguya dönüşmesidir. Analiz modülüne **baştan** konur.

Raporlanan metrikler

- Toplam mesafe (km)
- Yakıt tüketimi (L)
- Yakıt maliyeti (TL)
- Sürücü maliyeti (TL)
- **Toplam maliyet (TL)**
- Kullanılan araç sayısı
- Ortalama teslimat süresi
- ETA doğruluk sapması
- Hesaplama süresi (ms)
- **Tasarruf (%)**

Kırılımlar
Haftalık · aylık · yıllık — pandas resample ile optimization_runs.created_at üzerinden

Pazarlık payı olmayan kural

Sistem tek parça yazılmaz. Takvim sıkışırsa son çalışan katman sunulur; tek parça geliştirmede proje bitmezse ortada gösterilebilir hiçbir şey kalmaz.

1 KATMAN

Kargo listesi + adres doğrulama + tek araç rotası (TSP)

- Docker compose + PostGIS/pgRouting · Liquibase şema · seed filo
- OSM veri hattı: download → clip → load → export
- Kargo CRUD · haritadan konum seçimi · 5 katmanlı adres doğrulama · Dijkstra matris

Demo: "Paket ekliyorum, rota haritada yeniden çiziliyor."
Kapasite ve maliyet henüz yok. Tek araç, tek rota.
Risk: yol grafı kurulumu — en erken çözülmeli.

2 KATMAN

Çok araç + kapasite kısıtı + maliyet modeli (CVRP)

- Clarke-Wright savings → 2-opt / relocate / exchange / Or-opt → simulated annealing
- Amaç fonksiyonu: mesafe + yakıt + sürücü + öncelik · naive baseline kaydı
- **Olurluluk testleri** — kapasite ihlali asla üretilmemeli

Demo: "3 araçla dağıtım planı, TL cinsinden maliyet."
Naive planla farkı tablo halinde görünür.
Ölçüm altyapısı buradan itibaren hazır tutulur.

3 KATMAN

Maksimum rota süresi + ETA + bildirim + roller

- Çalışma günü süresi sert kısıt olarak algoritmaya girer
- Durak bazında ETA (maxspeed + servis süresi) · uygulama içi + e-posta bildirim
- JWT · üç rol · **rol sızıntısı testleri**

Demo: "Paketiniz tahmini 14:20'de teslim edilecek"
bildirimi alıcıya ulaşır; kurye kendi rotasını görür.

4 KATMAN

Dinamik yeniden rotalama

- Erteleme ("başka gün gelsin") · alıcı adres değişikliği · otomatik erteleme
- Teslim edilenler sabit, kalan duraklar yeniden optimize — son tamamlanan duraktan
- Beş güvenlik önlemi (sayfa 13) · sıcak başlangıç · debounce

Demo: Kurye yoldayken alıcı erteliyor,
rota canlı olarak yeniden hesaplanıyor.
Projenin "dinamik" iddiasının kanıtı.

5 KATMAN

Algoritma karşılaştırma modülü + istatistik raporları

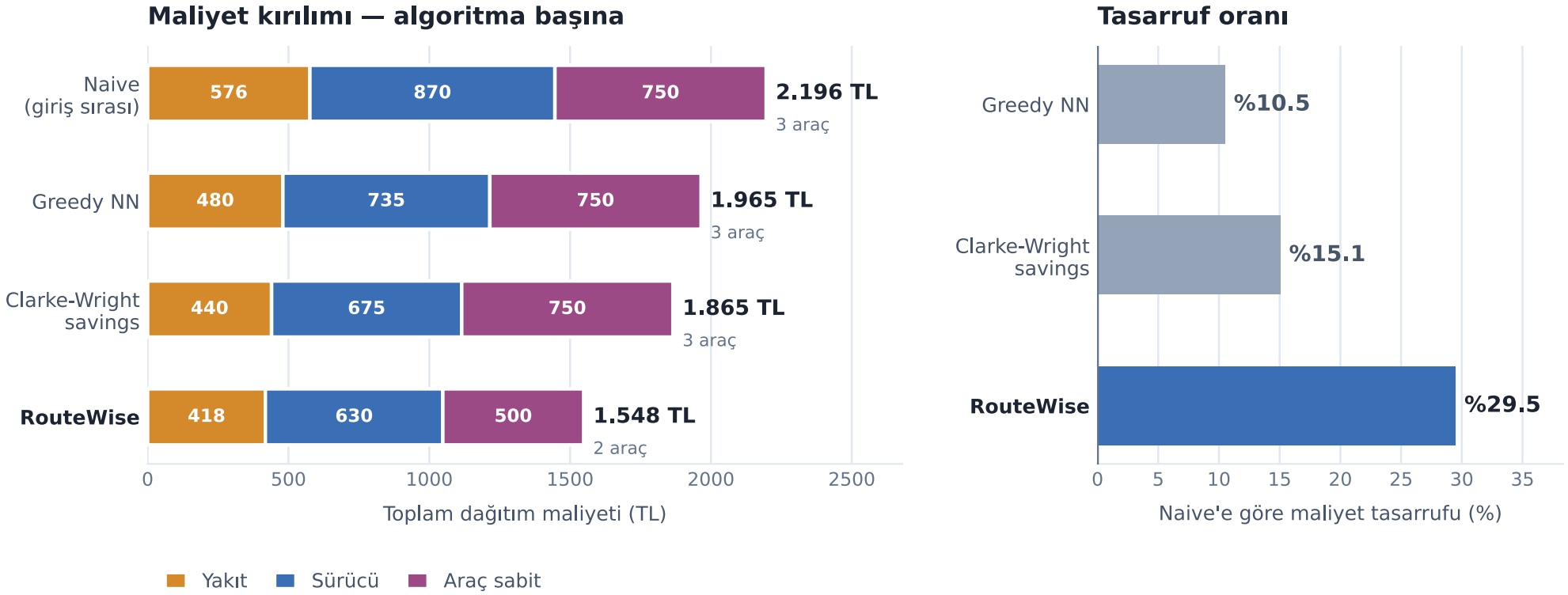
- Seviye 1: Dijkstra · A* · BFS · DFS — yol grafı üzerinde
- Seviye 2: naive · greedy · savings · RouteWise · DFS-exhaustive
- pandas/scipy analiz · Excel + PDF dışa aktarma · haftalık/aylık/yıllık tasarruf

Projenin en görünür çıktısı
Ölçüm altyapısı Katman 2'den beri hazır olduğu için
bu katman veri üretmeyi beklemez.

Algoritma maliyet karşılaştırması

ÖRNEK VERİ

Aynı kargo listesi, aynı filo, tek dağıtım günü · birim maliyetler: yakıt 44 TL/L · sürücü 150 TL/saat · araç 250 TL/gün



Figürü besleyen veri ÖRNEK

Algoritma	Yakıt TL	Sürücü TL	Araç TL	Toplam TL	Araç	Tasarruf
Naive (giriş sırası)	576	870	750	**2.196**	3	taban çizgi
Greedy NN	480	735	750	**1.965**	3	%10,5
Clarke-Wright savings	440	675	750	**1.865**	3	%15,1
RouteWise	418	630	**500**	**1.548**	**2**	**%29,5**

Birim maliyetler `cost_parameters` tablosundan gelir: yakıt 44 TL/L · sürücü 150 TL/saat · araç 250 TL/gün. **RouteWise aynı işi 2 araçla bitirdiği için araç sabit maliyeti düşer** — bu, amaç fonksiyonuna araç sabit maliyeti konulmasının somut etkisidir.

Bu sayfadaki sayılar örnektir. Figürün yapısını ve rapor biçimini göstermek için üretilmiştir; gerçek değerler `optimization_runs` tablosundan okunacaktır. Ölçüm henüz yapılmamıştır.

Nasıl üretiliyor

Figür `docs/diagrams/cost_comparison.py` betiğiyle **matplotlib** kullanılarak üretilir ve **vektörel SVG** olarak kaydedilir; teze ve sunuma kayıpsız gömülür. Betik, Python servisindeki `analytics/` modülünün üreteceği rapor figürünün prototipidir.

Renk paleti doğrulandı

Kategorik palet gözle değil, **çalıştırılabilir bir doğrulayıcıyla** seçildi: açıklık bandı, kroma tabanı, **renk körlüğü ayrımı** (protan/deutan/tritan) ve normal görme eşiği kontrollerinin tamamından geçer. İlk aday palet (mavi–mor) renk körlüğünde ΔE 3,0 ile **başarısız oldu** ve değiştirildi.

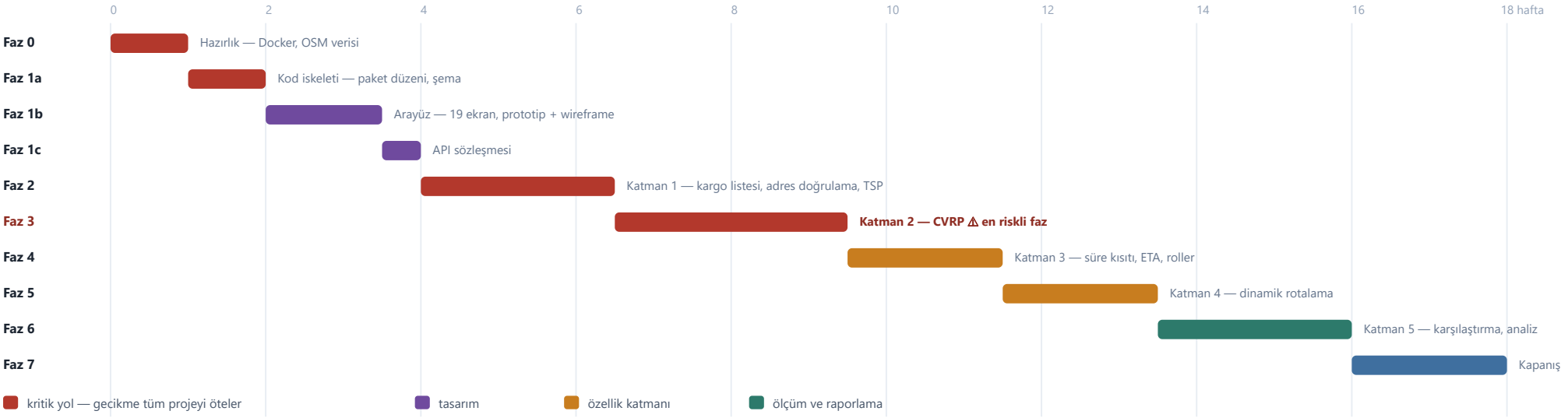
Kontrast uyarısı, her segmentin **doğrudan sayı etiketi** taşıması ve bu sayfadaki tablo ile karşılanır — renk tek başına bilgi taşımaz.

Raporda kullanılacak diğer figürler

20 Geliştirme Yol Planı — Genel Görünüm

8 faz · 14–18 hafta

Takvim



Temel ilke

Her fazın sonunda **çalışan ve demo edilebilir** bir sistem olur.

Takvim sıkışırsa son biten faz sunulur — hiçbir noktada elde yarım kalmış, gösterilemeyen bir sistem kalmaz.

Tek parça geliştirmede proje bitmezse ortada hiçbir şey kalmaz.

Kritik yol

Faz 0 → 1a → 2 → **3** · bu zincirde gecikme tüm projeyi öteler.

Faz 1b (arayüz) kısmen paralel yürütülebilir.

Faz 3 projenin kırılma noktasıdır: algoritma çalışmazsa proje çalışmaz.

Değişmeyen kurallar

1 Yarım bırakılmış katman üstüne yeni katman başlamaz.

2 **Olurluluk testleri algoritmadan önce yazılır.**
Sert kısıt ihlali sessizce geçerse sistem aylarca yanlış çalışır ve kimse fark etmez.

3 Ölçüm altyapısı Faz 3'ten itibaren hazırdır.
Faz 6, veri üretmeyi beklemez.

4 Python servisi veritabanına erişmez.
Hibrit mimarinin maliyetini düşüren tek şey budur.

5 Kapsam genişletme talepleri bir sonraki katmana yazılır, o anki katmanı bozmaz.
Örn. zaman penceresi, çok depo — kapsam dışı tutuldu.

21 Faz 0 — Hazırlık · Faz 1a — Kod İskeleti

2 hafta · kritik yolun başı

FAZ 0 — Hazırlık 1 hafta

Amaç: Geliştirme ortamı ayakta, Bandırma yol grafı veritabanında, `docker compose up` çalışıyor.

Görevler

- Docker Desktop'ı çalışır hale getir (`Start-Service com.docker.service`)
- Repo klasör yapısı: `backend/ python/ frontend/ infra/`
- `infra/docker-compose.yml` — 5 servis, 4 volume, healthcheck'ler
- `.env.example` yaz, `.env`'i gitignore'a ekle
- `infra/dataprep/` imajı — osmium + osm2pgrouting
- **Bandırma sınır poligonunu üret** — OSM ilişki kimliğinden `.poly`
- Veri hattı: download → clip → load → export

Biten iş tanımı

- ☐ Dört servis ayağa kalkıyor, healthcheck'ler yeşil
- ☐ `ways` tablosunda Bandırma yol ağı var
- ☐ `pgr_dijkstra` örnek sorgusu mesafe döndürüyor
- ☐ `graphdata` volume'ünde graf dosyası var
- ☐ **Başka biri sıfırdan kurabiliyor** (README)

Risk: Poligon yanlışsa graf boş çıkar — `ways` satır sayısına ve haritada birkaç yola bak. Docker motoru bu makinede bilinen sorun; Faz 0'ın ilk maddesi olmasının sebebi bu.

FAZ 1a — Kod İskeleti 1 hafta

Amaç: Üç proje de derleniyor, paket düzeni oturmuş, boş katmanlar yerinde. **Henüz iş mantığı yok.**

Backend paket düzeni

```
com/routewise/
  config/     Security · Jackson · OpenApi
  common/     error/ validation/ geo/
  auth/       JWT · login · rol
  user/ courier/ recipient/
  depot/ vehicle/
  address/
    verification/ durum makinesi · geofence
  shipment/   sipariş no üretimi
  plan/
  routing/
    matrix/   Provider · cache · Dijkstra
    graph/    pgRouting referansı
  optimization/
    client/   PythonOptimizerClient
    service/  koşu yönetimi
  comparison/ analytics/ notification/
```

Paket-özellik düzeni: her özellik kendi entity/repository/service/controller katmanını içinde taşır. 13 tablolu bir projede katman-bazlı düzenden okunaklıdır.

Python modül düzeni

```
app/
  main.py     açılışta graf yükleme
  schemas/    pydantic modelleri
  graph/      model.py loader.py
  pathfinding/
    dijkstra · astar · bfs · dfs
  vrp/
    model.py objective.py
    feasibility.py
    construction/ savings · nn · naive
    localsearch/  2opt · relocate
                  exchange · or_opt
    metaheuristic/ simulated_annealing
    exhaustive.py routewise.py
  analytics/
    aggregate significance
    figures export
  api/  optimize compare analytics
```

Frontend

```
src/
  lib/  apiClient authStore
  types/ zod şemaları
  components/ ui/ map/ charts/ layout/
  features/
    auth shipments addresses
    verification planning routes
    comparison reports
    courier recipient
```

Biten iş tanımı

- ☐ backend ve python healthcheck'leri 200
- ☐ Liquibase **13 tablonun tamamını** kuruyor
- ☐ `down && up` tekrarlanabilir
- ☐ Python gerçek grafı yükleyip düğüm sayısını logluyor
- ☐ Frontend backend'den yanıt alıyor
- ☐ Testler boş da olsa koşuyor

22 Faz 1b — Arayüz Tasarımı · Faz 1c — API Sözleşmesi

2 hafta · 19 ekran

Firma — kargo şirketi

#	Ekran	İçerik
F1	Giriş	e-posta + parola
F2	Gösterge paneli	günün özeti, bekleyen doğrulama
F3	Kargo listesi	filtre, durum rozetleri
F4	Yeni kargo	alıcı, ağırlık, öncelik
F5	**Adres doğrulama**	harita önizleme, onayla/reddet
F6	Rota planlama	harita, optimize et, ilerleme
F7	Rota detayı	durak sırası, ETA, maliyet
F8	Karşılaştırma	Seviye 1 ve 2 tabloları
F9	Tasarruf raporları	grafikler, Excel/PDF indirme

F5 iş kuralı en yoğun ekrandır: beş katmanlı adres doğrulamanın insan onayı adımı burada.

Kurye

#	Ekran	İçerik
K1	Giriş	—
K2	Bugünkü rotam	sıralı duraklar, ETA, toplam km
K3	Durak detayı	teslim edildi / ulaşılamadı
K4	Harita görünümü	sıradaki durak vurgulu
K5	**Rota değişti**	neyin değiştiği açıkça

Alıcı

#	Ekran	İçerik
A1	Giriş	—
A2	Kargolarım	durum, tahmini teslim saati
A3	Kargo detayı	ETA, "başka gün gelsin"
A4	**Adresim**	haritadan düzenleme + durum rozeti
A5	Bildirimler	ETA, erteleme, doğrulama

Risk: Ekran tasarımı gereksinim açığı ortaya çıkarır — bu iyidir, ama mimariye ve veri modeline geri dönmeyi gerektirebilir. Bunun için zaman payı bırakıldı.

Faz 1b — görevler

- Ekran envanterini **dondur** (19 ekran)
- Her ekran için durum varyantları: **boş, yükleniyor, hata, dolu**
- Kritik akışları çiz: kargo oluşturma → doğrulama → planlama → teslim; adres değiştirme → yeniden rotalama
- **Tıklanabilir HTML prototip** üret — ekranlar arası geçiş çalışır
- Prototipten **PDF wireframe** türet

☐ 19 ekranın tamamı prototipte ve birbirine bağlı
☐ Üç rolün ana akışı baştan sona tıklanabiliyor
☐ Her ekranın boş/hata durumu tasarlanmış
☐ PDF wireframe `docs/` altında
☐ **Eksik gereksinim kalmadığı onaylandı**

FAZ 1c — API Sözleşmesi 0,5 hafta

- Ekranlardan **gereken** endpoint'leri türet — fazlasını yazma
- Her endpoint: yol, metot, **yetki rolü**, istek/yanıt gövdesi, hata kodları
- Ortak hata biçimi: `ApiError { code, message, fieldErrors[] }`
- Sayfalama, sıralama, filtreleme kuralları
- OpenAPI/Swagger üretimini aç
- Frontend tiplerini zod şemalarıyla eşle

☐ Her ekranın ihtiyacı bir endpoint'le karşılanıyor
☐ **Karşılıksız endpoint yok**
☐ Yetki matrisi her endpoint'e uygulanmış
☐ Swagger arayüzü açılıyor
☐ Postman collection çıkarıldı

23 Faz 2 — Katman 1 · Faz 3 — Katman 2 (CVRP)

5,5 hafta · kritik yol

FAZ 2 — Kargo listesi, adres doğrulama, tek araç 2,5 hafta

Amaç: Kargo ekleyip çıkarabiliyoruz; adres haritadan seçiliyor, beş katmanlı kontrolden geçiyor, firma doğruluyor; tek araç için rota hesaplanıp haritada çiziliyor.

Görevler

Katman	İş
Backend	Auth + JWT + üç rol
Backend	Kargo CRUD + sipariş no üretimi `BND-YYYY-NNNNN`
Backend	**Beş katmanlı adres doğrulama:** geofence · yola snap · ters geocoding · aykırı değer · firma onayı
Backend	`DistanceMatrixProvider` + Dijkstra + matris cache
Python	Graf yükleyici + `dijkstra.py`
Python	Tek araç TSP (nearest neighbor ile başla)
Python	`POST /optimize` ilk sürüm
Frontend	F1, F3, F4, F5 + haritadan konum seçimi
Frontend	Rota haritada çizimi

Biten iş tanımı

- ☐ Bandırma dışına tıklanınca **reddediliyor**
- ☐ Yoldan uzağa tıklanınca uyarı, snap mesafesi kaydediliyor
- ☐ Doğrulanmamış adresli kargo rotaya **girmiyor**
- ☐ Alıcı adresi değiştirince durum otomatik geri düşüyor
- ☐ Kendi Dijkstra'mız `pgr_dijkstra` ile **aynı mesafeyi** veriyor
- ☐ Matris cache çalışıyor — aynı küme ikinci kez hesaplanmıyor
- ☐ 10 teslimatlık liste haritada sıralı rota olarak çiziliyor

Risk: Snap eşiği yanlış ayarlanırsa geçerli adresler reddedilir veya geçersizler kabul edilir — Bandırma'dan gerçek 20–30 adresle elle sına.
Dijkstra doğrulaması tutmazsa graf yüklemesinde hata vardır; sonraki her şey buna dayandığı için burada durulmalı.

FAZ 3 — CVRP 3 hafta ⚠ en riskli faz

Amaç: Çok araç, kapasite sert kısıtı, tam maliyet modeli, RouteWise algoritması çalışıyor.

Görevler

#	İş
1	`vrp/model.py` — Solution, Route, Stop
2	`objective.py` — mesafe + yakıt + sürücü + öncelik + araç sabit
3	**`feasibility.py` — kapasite kontrolleri (ÖNCE yaz)**
4	`construction/savings.py` — Clarke-Wright, kapasite farkında
5	`localsearch/` — 2-opt, relocate, exchange, Or-opt
6	`simulated_annealing.py` — tohumlanmış, sıcak başlangıçlı
7	`routewise.py` — aşamaları birleştir
8	`construction/naive.py` — baseline, her koşuda birlikte
9	Backend: koşu + rota + durak kalıcılaştırma
10	Frontend: F6, F7 — planlama, maliyet kırılımı

Biten iş tanımı

- ☐ **Hiçbir hamle kapasite ihlali üretmiyor** (olurluluk testleri yeşil)
- ☐ Aynı tohumla aynı girdi **aynı rotayı** veriyor
- ☐ Sıcak başlangıç: tek paket eklenince rota baştan aşağı değişmiyor
- ☐ Naive baseline her koşuda kaydediliyor
- ☐ n ≤ 8'de sonuç, tam aramanın optimumuna makul yakın
- ☐ Maliyet kırılımı ekranda TL olarak görünüyor

Bu projenin kırılma noktasıdır. Algoritma çalışmazsa proje çalışmaz.

Olurluluk testleri önce yazılmalı, sonra algoritma. Aksi hâlde sessizce geçersiz rota üreten bir sistemle aylarca çalışılır.

SA parametreleri (başlangıç sıcaklığı, soğuma oranı, iterasyon sayısı) ayar gerektirir — bunun için zaman payı ayrıldı.

24 Faz 4 – 7 · Katman 3–5 ve Kapanış

8,5 hafta

FAZ 4 — Süre kısıtı, ETA, bildirim, roller 2 hafta

- `feasibility.py`'ye **maksimum rota süresi** kısıtını ekle
- Süre hesabı: segment uzunluğu / `maxspeed` + servis süresi
- ETA hesabı ve `route_stops.eta` doldurma
- Bildirim altyapısı: uygulama içi + Spring Mail
- Rol bazlı yetkilendirme tüm endpoint'lerde
- Kurye ekranları K1–K4 · Alıcı ekranları A1–A5

☐ Gün sınırını aşan çözüm **üretilmiyor**
☐ ETA'lar makul (elle birkaç durak doğrula)
☐ Alıcı e-posta ve uygulama içi bildirim alıyor
☐ **Rol sızıntısı testleri yeşil** — alıcı başkasının teslimatını göremiyor
☐ Kurye yalnızca kendi rotasını görüyor

FAZ 5 — Dinamik yeniden rotalama 2 hafta

- "Başka gün gelsin" → erteleme → kümeden çıkarma → yeniden optimizasyon
- Dağıtım sırasında adres değişikliği → doğrulama → yeniden rotalama
- **Son tamamlanan duraktan** yeniden optimizasyon (canlı GPS yok)
- Beş güvenlik önlemi: otomatik erteleme · `YOLDA` adres kilidi · maliyet eşiği · günlük limit · kuryeye bildirim
- Debounce (~500 ms) + asenkron koşu + ilerleme göstergesi

☐ Erteleme sonrası teslim edilenler sabit kalıyor
☐ Gün sınırı aşılırsa teslimat **otomatik erteleniyor**
☐ `YOLDA` teslimatın adresi değiştirilemiyor
☐ Kurye ekranında eski durak kalkıyor, bildirim düşüyor
☐ Arka arkaya 10 değişiklikte **1** optimizasyon koşuyor

FAZ 6 — Karşılaştırma ve analiz 2,5 hafta

Grup	İş
Seviye 1	`astar.py` · `bfs.py` · `dfs.py`
Seviye 1	Metrik toplama: uzunluk, süre, gezilen düğüm
Seviye 1	`POST /compare/path` + kayıt tablosu
Seviye 2	`nearest_neighbor.py` · `exhaustive.py` (n ≤ 10)
Seviye 2	`POST /compare/tour` — N kez koş, medyan al
Analiz	`aggregate.py` — haftalık/aylık/yıllık resample
Analiz	**`significance.py` — Wilcoxon / t-test**
Analiz	`figures.py` — matplotlib figürleri
Analiz	`export.py` — Excel (5 sayfa) + PDF
Frontend	F8, F9 — tablolar, Recharts, indirme

☐ BFS'in ağırlıkları yok saydığı **sayısal olarak** gösteriliyor
☐ A* aynı yolu daha az düğüm gezerek buluyor
☐ DFS-exhaustive küçük örnekte gerçek optimumu veriyor
☐ "RouteWise optimumun %X içinde" veriyle destekleniyor
☐ **En az 30 senaryoda** anlamlılık testi, p değeri raporlanıyor
☐ Excel ve PDF raporları indirilebiliyor

Risk: Yeterli senaryo üretilmezse anlamlılık testi anlamsız kalır. Rastgele teslimat kümesi üreten senaryo üretici bu fazda yazılmalı.

FAZ 7 — Kapanış 2 hafta

- Test boşluklarını kapat — satır ≥ %80, dal ≥ %70
- Linter/formatter temiz: spotless · ruff + mypy · eslint + prettier
- `README.md` — sıfırdan kurulum, ekran görüntüleri
- `architecture.md`'yi son hâle güncelle
- Tez raporu: figürler, tablolar, sonuç bölümü
- Demo senaryosu provası · bilinen eksikler listesi

☐ Temiz derleme, uyarısız · tüm testler yeşil
☐ Coverage hedefleri tutuyor
☐ **Başka bir makinede** sıfırdan kurulum denendi
☐ Demo senaryosu baştan sona provalandı

25 Genel Riskler ve Önlemler

RİSK	ERKEN UYARI	ÖNLEM
Algoritma beklenen kaliteyi vermiyor Faz 3'ün kırılma noktası olmasının sebebi	Faz 3 sonunda küçük örnekte optimuma uzak kalıyor	SA parametre ayarı için zaman payı; gerekirse komşuluk yapısını genişlet
Kapsam genişlemesi zaman penceresi, çok depo, SMS…	"Şunu da ekleyelim" konuşmaları	Katmanlı plana sadık kal; yeni istek bir sonraki katmana yazılır
Yol grafı sorunları poligon yanlış, import eksik	Faz 0'da `ways` satır sayısı tuhaf görünüyor	Poligonu ve import loglarını kontrol et, **Faz 0'dan çıkma**
Takvim sıkışması	Faz 4 sonunda Faz 5'e vakit kalmıyor	Son çalışan katmanı sun — her faz demo edilebilir olduğu için bu güvenli
Sessiz geçersiz rotalar en sinsi risk	**Olurluluk testi yoksa hiç fark edilmez**	Testleri algoritmadan **önce** yaz (değişmeyen kural #2)
Docker motoru çalışmıyor	`docker info` hata veriyor	Faz 0'ın ilk maddesi — bu makinede bilinen sorun

En sinsi risk: sessiz geçersiz rotalar

Kapasite veya süre kısıtı bir yerel arama hamlesinde sessizce ihlal edilirse, sistem **çalışıyor gibi görünür**: rota çizilir, maliyet hesaplanır, rapor üretilir — ama rota uygulanamaz.

Bu hata test edilmeden fark edilmez.

Kullanıcı davranışı test ederek doğrular, kodu okuyarak değil; geçen bir test ise yalnızca dokunduğu yolu kanıtlar.

Önlem: olurluluk testleri algoritmadan önce yazılır ve her hamle tipi için ayrı ayrı sınanır.

Faz 3'ün görev listesinde 3. madde, algoritmadan önce gelir.

Faz bağımlılıkları

Faz 0 → 1a → 2 → 3 kritik yol, sıkı bağımlı

Faz 1b (arayüz) Faz 1a ile **paralel** yürütülebilir

Faz 1c, Faz 1b bitmeden tamamlanamaz

Faz 4 ve 5, Faz 3'ün çıktısına dayanır

Faz 6, Faz 3'ten itibaren biriken **ölçüm verisine** dayanır

— bu yüzden ölçüm altyapısı Faz 3'te kurulur, Faz 6'da değil

Toplam tahmin: 14–18 hafta
Süreler kendi takvimine göre esnetilmelidir.